

STEP 2 · 2주차

전처리와 특징 추출

원신호를 25차원 숫자 벡터로 바꾼다

강의 50분

용어 정리

실습 6단계

이번 주는 딥러닝이 없다. 사람이 직접 정의한 통계량을 손으로 만들어 본다.

이번 주 진행 방식

지난주와 같은 순서로

① 강의

50분

전처리 · 시간영역 특징 5종

본 강의 자료 16~30페이지

② 용어 정리

20분

필터부터 표준화까지

이 자료 4~7페이지

③ 실습

70분

필터 → 윈도우 → 특징 → 데이터셋

이 자료 9~14페이지

④ 과제

다음 주까지

25차원 특징 데이터셋 완성

이 자료 17페이지

지난주 과제를 끝내지 못했다면 먼저 그것부터. 데이터를 읽지 못하면 오늘 실습을 시작할 수 없다.

오늘 강의에서 기억할 여섯 가지

실습 전에 다시 확인

- | | | |
|---|------------|---|
| 1 | 두 개의 필터 | 60Hz 노치로 전기 잡음을, 20~450Hz 대역통과로 근활성 대역만 남긴다 |
| 2 | 2초 세그먼트 | 2초(2,000샘플) 분석 구간을 쓴다. 동작 수행은 1초다 |
| 3 | 200ms 윈도우 | 2초를 200ms씩 100ms 간격으로 자르면 19개 조각이 나온다 |
| 4 | 시간영역 특징 5종 | RMS · MAV · WL · ZC · SSC — 계산이 가볍고 견고하다 |
| 5 | 25차원 벡터 | 특징 5종 × 채널 5개 = 윈도우 하나당 25개 숫자 |
| 6 | 표준화 | StandardScaler로 표준화한 특징을 세 모델에 사용한다 |

오늘 실습은 이 여섯 단계를 순서대로 코드로 옮기는 일이다. 순서가 곧 코드의 구조다.

용어 정리 ① 필터

원하지 않는 성분 걸러내기

주파수

Frequency

1초에 몇 번 진동하는가 · 단위 Hz

→ 60Hz = 콘센트에서 들어오는 잡음

노치 필터

Notch filter

특정 주파수 하나만 좁게 제거

→ 60Hz 전원선 간섭 제거

대역통과 필터

Band-pass filter

정해진 구간만 통과시킴

→ 20~450Hz만 남긴다

차수

Order

얼마나 가파르게 잘라내는가

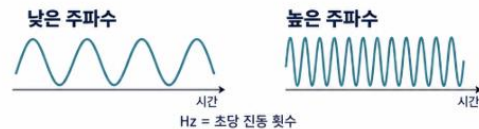
→ 4차 Butterworth 사용

영위상 필터링

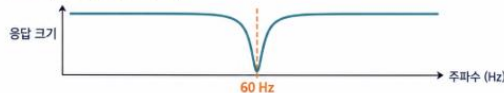
filtfilt

앞뒤 양방향으로 걸어 지연을 없앴

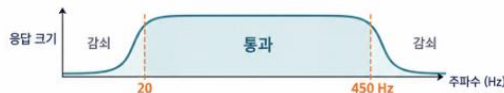
→ 신호가 밀리지 않는다



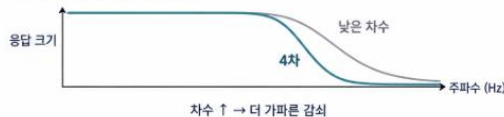
노치: 60 Hz 억제



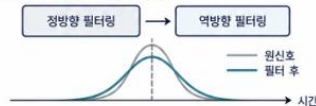
대역통과: 20~450 Hz



4차 Butterworth



영위상 필터링 · firlfilt



논문 조건: 60Hz 노치 + 20~450Hz 4차 Butterworth. 노치 Q와 적용 방식은 실습 설정이다.

용어 정리 ② 자르기

긴 신호를 학습 단위로

세그먼트

Segment

의미 있는 한 덩어리 구간

→ 분석 구간 2초 = 2,000샘플

윈도우

Window

분석 단위로 잘라낸 조각

→ 200ms = 200샘플

증분 / 스텝

Increment · Step size

다음 조각으로 이동하는 간격

→ 100ms = 50% 오버랩

오버랩

Overlap

옆 조각과 겹치는 비율

→ 50% — 조각 수가 약 두 배

윈도우 수

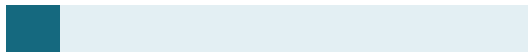
—

한 세그먼트에서 나오는 조각 수

→ 2초에서 19개

슬라이딩 윈도우

2초 = 2,000샘플



한 창: 200샘플

창 1

창 2

창 3

100샘플씩 이동 · 50% 겹침
총 19개 창 × 창마다 25D

$(2000 - 200) / 100 + 1 = 19$. 이 계산을 직접 해보면 윈도우 개념이 확실해진다.

용어 정리 ③ 시간영역 특징

채널당 5개 특징 · 5채널에서 25차원

RMS

Root Mean Square

제곱 평균의 제곱근 — 신호 파워

→ 근수축 강도를 나타낸다

MAV

Mean Absolute Value

절댓값의 평균

→ 가장 널리 쓰이는 기본 특징

WL

Waveform Length

인접 샘플 차이의 누적 합

→ 파형의 복잡도

ZC

Zero Crossing

0을 지나는 횟수 (임계값 0.01)

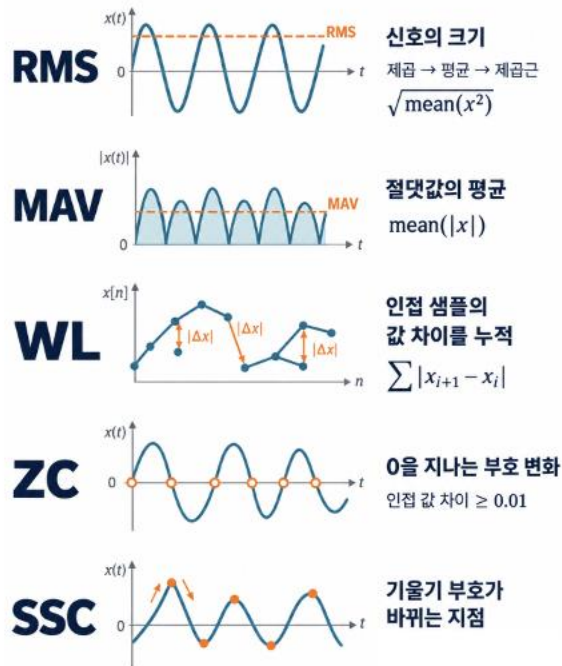
→ 주파수 성분의 간접 지표

SSC

Slope Sign Change

기울기 부호가 바뀌는 횟수

→ 고주파 정보를 보완



ZC의 임계값 0.01은 잡음에 의한 미세한 부호 변화를 세지 않기 위한 장치다.

용어 정리 ④ 표준화와 데이터셋

모델에 넣기 직전 처리

특징 벡터

Feature vector

한 샘플을 나타내는 숫자들의 나열

→ 25차원 = 5특징 × 5채널

표준화

Standardization

평균 0 · 분산 1로 맞추기

→ StandardScaler 사용

fit / transform

-

통계를 계산 / 그 통계를 적용

→ 학습셋에 fit, 나머지에 transform

층화 분할

Stratified

클래스 비율을 유지하며 나누기

→ 8클래스가 고르게 들어가게

데이터 누수

Data leakage

평가 정보가 학습에 새어 들어감

→ 같은 시행의 겹친 창이 양쪽에 섞이면 발생

표준화 통계는 반드시 학습셋에서만 계산한다. 테스트셋 통계를 쓰면 정보 누출이다.

1 25차원 특징 벡터



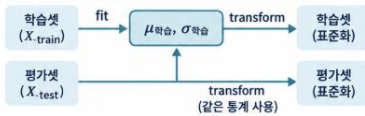
2 표준화

$$z = \frac{x - \mu_{\text{학습}}}{\sigma_{\text{학습}}}$$

학습셋의 특징별 평균 0 · 분산 1
비상수 특징 기준



3 fit / transform



학습셋: fit → transform | 평가셋: 같은 통계로 transform

4 층화 분할



5 데이터 누수 방지



같은 시행의 창은 한쪽에만

오늘의 실습

여섯 단계 — 앞 단계 결과가 다음 단계 입력이 된다

단계 1	필터 적용	60Hz 노치 + 20~450Hz 대역통과
단계 2	분석 구간 잘라내기	2초 분석 구간을 추출한다
단계 3	슬라이딩 윈도우	2초를 19개 조각으로
단계 4	시간영역 특징 구현	RMS·MAV·WL·ZC·SSC 직접 구현
단계 5	데이터셋 만들기 — 누수 막기	시행 단위로 먼저 나눈다
단계 6	표준화와 저장	StandardScaler 적용 후 파일로 저장

한 단계 끝날 때마다 결과를 파일로 저장할 것. 중간에 막혀도 처음부터 다시 하지 않아도 된다.

필터 적용

논문 필터 조건을 적용한 실습 예시

목표 원신호에서 전원선 잡음과 대역 밖 성분을 제거한다

```
import numpy as np
from scipy.signal import butter, iirnotch, filtfilt
FS = 1000

def preprocess(x, fs=FS):
    bn, an = iirnotch(60, 30, fs)
    x = filtfilt(bn, an, x, axis=0)
    b, a = butter(4, [20/(fs/2), 450/(fs/2)], btype='band')
    return filtfilt(b, a, x, axis=0)

xf = preprocess(x)
print('전:', x.std(0).round(4))
print('후:', xf.std(0).round(4))
```

확인 방법

- 채널 5개의 표준편차가 각각 출력되면 성공
- axis=0 을 빼면 채널축에 걸린다
- 필터 전후 파형과 표준편차를 함께 확인
- 노치 Q=30과 filtfilt는 실습 설정이다

분석 구간 잘라내기

논문은 2초 구간의 정확한 시작점을 명시하지 않는다

목표 실습 시작점을 확인하고 2초를 세그먼트로 추출한다

```
# 시행 구조: 휴식 2초 | 동작 1초 | 휴식 2초
# 아래 1.5초 시작은 실습 가정이며 논문 설정이 아님
START, LEN = 1500, 2000            # 1.5초 지점부터 2초
```

```
def cut(x, start=START, n=LEN):
    return x[start:start+n]
```

```
seg = cut(xf)
print('세그먼트 모양:', seg.shape)    # (2000, 5)
```

확인 방법

- (2000, 5) 가 나오면 정상
- 시작 지점은 데이터에 맞게 조정할 것
- 지난주 그림에서 동작 구간 위치를 확인해 두었을 것
- 자른 구간을 다시 그려 확인하는 습관

슬라이딩 윈도우

200ms · 100ms 간격

목표 세그먼트를 200ms 길이로 100ms씩 밀며 잘라낸다

```
WIN, HOP = 200, 100

def make_windows(x, win=WIN, hop=HOP):
    n = (len(x) - win) // hop + 1
    return np.stack([x[i*hop : i*hop+win] for i in range(n)])

w = make_windows(seg)
print('윈도우 배열:', w.shape)      # (19, 200, 5)
```

확인 방법

- (19, 200, 5) 가 나오면 정상
- $19 = (2000 - 200) / 100 + 1$
- 숫자가 다르면 세그먼트 길이를 다시 확인
- 19개는 서로 겹친다 — 단계 6에서 중요해진다

시간영역 특징 구현

라이브러리 없이 numpy만으로

목표 윈도우 하나에서 25차원 특징 벡터를 만든다

```
def td_features(w, th=0.01):
    """w: (200, 5) -> (25,)"""
    rms = np.sqrt(np.mean(w**2, axis=0))
    mav = np.mean(np.abs(w), axis=0)
    d = np.diff(w, axis=0)
    w1 = np.sum(np.abs(d), axis=0)
    zc = np.sum(((w[:-1]*w[1:]) < 0) &
                (np.abs(d) >= th), axis=0)
    ssc = np.sum((d[:-1]*d[1:]) < 0, axis=0)
    return np.concatenate([rms, mav, w1, zc, ssc])

f = td_features(w[0])
print('특징 벡터:', f.shape)      # (25,)
```

확인 방법

- (25,) 가 나오면 성공
- 순서를 문서에 기록할 것 — 나중에 해석에 필요
- NaN 이 나오면 입력에 이상값이 있는지 확인
- th 를 0으로 바꿔보고 ZC 값이 어떻게 변하는지 관찰

데이터셋 만들기 — 누수 막기

윈도우를 표본으로 쓰는 실습 예시 · 논문은 창당 25D 명시

목표 시행 단위로 분할한 뒤 각 집합 안에서만 윈도우와 특징을 만든다

```
from sklearn.model_selection import train_test_split

# 1) 시행(파일) 단위로 먼저 분할 ← 순서가 핵심
tr_f, te_f = train_test_split(
    files, test_size=0.2, stratify=labels,
    random_state=42)

# 2) 각 집합 안에서만 특징 생성
def build(flist):
    X, y = [], []
    for p in flist:
        seg = cut(preprocess(np.loadtxt(p, delimiter=',')))
        for wnd in make_windows(seg):
            X.append(td_features(wnd)); y.append(label_of(p))
    return np.stack(X), np.array(y)

Xtr, ytr = build(tr_f)
Xte, yte = build(te_f)
print(Xtr.shape, Xte.shape)
```

확인 방법

- 순서를 바꾸면 안 된다 — 윈도우 먼저 만들고 나누면 누수
- stratify 로 8클래스가 양쪽에 고르게 들어간다
- random_state 고정으로 다음에도 같은 분할
- 학습셋 샘플 수가 테스트의 약 4 배여야 정상

표준화와 저장

다음 주에 바로 쓸 수 있게

목표 학습셋 통계로 표준화하고 데이터셋을 파일로 남긴다

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
Xtr_s = sc.fit_transform(Xtr)      # 학습셋에서 fit
Xte_s = sc.transform(Xte)         # 테스트엔 적용만

print('학습셋 평균:', Xtr_s.mean(0).round(6))
print('학습셋 표준편차:', Xtr_s.std(0).round(6))

np.savez('dataset.npz', Xtr=Xtr_s, ytr=ytr,
         Xte=Xte_s, yte=yte)
print('저장 완료')
```

확인 방법

- 비상수 특징별 평균 0 · 표준편차 1 확인
- 테스트셋은 정확히 0.1이 아니어도 정상이다
- fit_transform 을 테스트셋에 쓰면 정보 누출
- dataset.npz 파일이 생성되었는지 확인

여기까지 됐으면 정상

스스로 확인하는 체크포인트

오늘 안에 확인할 것

- 필터 적용 후 5채널 표준편차가 출력된다
- 세그먼트가 (2000, 5) 로 잘린다
- 윈도우 배열이 (19, 200, 5) 로 나온다
- 특징 벡터가 (25,) 로 나온다
- 시행 단위로 먼저 분할한 뒤 특징을 만들었다
- 학습셋 평균 0 · 분산 1을 확인했다
- dataset.npz 파일이 저장되었다

7번까지 됐다면 이번 주 목표는 달성이다. 모델 학습은 다음 주에 한다.

자주 나는 오류와 해결

대부분 여기서 막힌다

증상	원인	해결
critical frequencies 오류	대역 상한이 나이퀴스트 이상	450 이하 값 사용 · fs 확인
윈도우 개수가 19가 아님	세그먼트 길이가 2000이 아님	cut 의 START·LEN 을 다시 계산
특징에 NaN 이 섞임	입력 결측치·무한값 또는 수치 오류	np.isnan(X).sum() 으로 위치 확인
정확도가 비현실적으로 높음	같은 시행의 겹친 창이 양쪽에 섞였는지 확인	단계 5의 순서를 다시 확인
표준화 후 값이 이상함	테스트셋에 fit_transform 사용	transform 으로 변경

네 번째 줄이 가장 중요하다. 결과가 너무 좋으면 기뻐하기 전에 누수부터 의심할 것.

이번 주 제출물

다음 주 강의 전까지

① 필터 검증

필터 전후 스펙트럼 비교 그림

② 윈도우 확인

윈도우 배열 모양 출력 화면

③ 특징 구현 코드

td_features 함수 전체

④ 분할 코드

시행 단위로 먼저 나눈 부분을 표시

⑤ 데이터셋 요약

학습·테스트 샘플 수 · 클래스별 분포

⑥ 설정 근거

윈도우 200ms · 오버랩 50%를 고른 이유 각 두 문장

제출 형식

노트북 또는 PDF 1개 · dataset.npz · 저장소 링크

④번을 반드시 표시할 것. 데이터 누수 여부는 이 부분만 보고 판단한다.

다음 주에 할 일

3주차 미리보기

다음 주 강의 내용

- SVM · Random Forest · XGBoost
- hold-out과 5-fold 교차검증
- 평가 지표 네 가지
- 대응표본 t-검정

다음 주 실습에서 만들 것

- 세 모델의 성능 비교표
- 5-fold 평균 $\pm$ 표준편차
- 8클래스 혼동행렬
- 통계 검정 결과

미리 해두면 좋은 것

- dataset.npz 를 반드시 저장해 둘 것 — 다음 주 실습은 이 파일에서 시작한다
- scikit-learn 의 SVC · RandomForestClassifier 문서를 훑어볼 것
- 시간이 남으면 세 모델 중 하나를 미리 돌려봐도 좋다



DAEGU
CATHOLIC
UNIVERSITY



Thank you



[Http://dil.cu.ac.kr/](http://dil.cu.ac.kr/) 데이터 지능 연구실